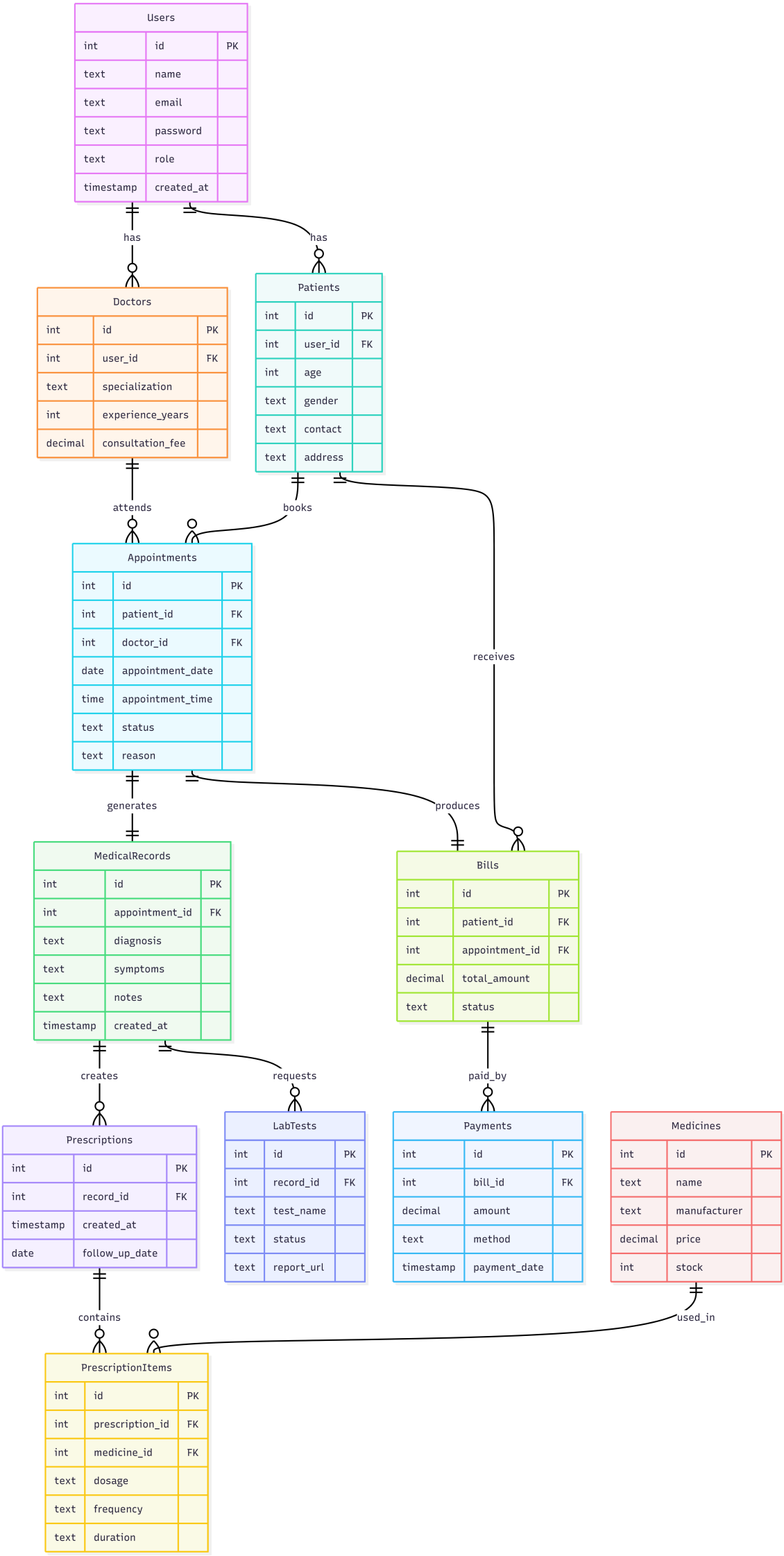
Users
int id PK
text name
text email
text password
text role
timestamp created_at
has
has
Doctors
int id PK
int user_id FK
text specialization
int experience_years
decimal consultation_fee
Patients
int id PK
int user_id FK
int age
text gender
text contact
text address
attends
books
receives
Appointments
int id PK
int patient_id FK
int doctor_id FK
date appointment_date
time appointment_time
text status
text reason
generates
produces
MedicalRecords
int id PK
int appointment_id FK
text diagnosis
text symptoms
text notes
timestamp created_at
Bills
int id PK
int patient_id FK
int appointment_id FK
decimal total_amount
text status
creates
requests
paid_by
Prescriptions
int id PK
int record_id FK
timestamp created_at
date follow_up_date
LabTests
int id PK
int record_id FK
text test_name
text status
text report_url
Payments
int id PK
int bill_id FK
decimal amount
text method
timestamp payment_date
Medicines
int id PK
text name
text manufacturer
decimal price
int stock
contains
used_in
PrescriptionItems
int id PK
int prescription_id FK
int medicine_id FK
text dosage
text frequency
text duration

🏥 Hospital Appointment and Management System

🔷 User Roles

1. Patient (User)

- Registers and logs into system
- Books appointments with doctors
- Views appointment status
- Views consultation records
- Views prescriptions and lab reports
- Views billing and payment details

---

2. Doctor

- Logs into system
- Views assigned appointments
- Views patient details
- Updates appointment status
- Adds consultation notes
- Generates prescriptions
- Requests lab tests

---

3. Admin

- Logs into system
- Adds and manages doctors
- Views users and system data
- Monitors appointments
- Manages system access

◈ System Modules

👤 User (Patient) Module

Tables Used:

- Users
- Patients
- Appointments
- MedicalRecords
- Prescriptions
- Bills

Functionalities

- User registration
- User login authentication
- Profile management
- Book appointment with doctor
- View appointment status
- View consultation history
- View prescriptions
- View billing information

🧑‍⚕️ Doctor Module

Tables Used:

- Users
- Doctors
- Appointments

- MedicalRecords
- Prescriptions
- LabTests

Functionalities

- Doctor login
- View assigned appointments
- View patient details
- Update appointment status
- Add diagnosis and consultation notes
- Generate prescriptions
- Request lab tests

🗂️ Admin Module

Tables Used:

- Users
- Doctors
- Patients
- Appointments

Functionalities

- Admin login
- Add new doctors
- Manage doctor details
- View registered users
- View all appointments
- Monitor system activity

📅 Appointment Management Module

Table:

- Appointments

Functionalities

- Appointment booking
- Appointment scheduling
- Appointment status tracking
- Doctor assignment
- Patient–doctor mapping

🗒 Medical Record Management Module

Table:

- MedicalRecords

Functionalities

- Store consultation details
- Maintain patient medical history
- Record symptoms and diagnosis
- Link appointment with treatment

💊 Prescription Management Module

Tables:

- Prescriptions
- PrescriptionItems
- Medicines

Functionalities

- Create prescriptions

- Assign medicines
- Store dosage instructions
- Manage medicine data

🧪 Lab Test Module

Table:

- LabTests

Functionalities

- Request lab tests
- Track test status
- Store test reports

💳 Billing & Payment Module

Tables:

- Bills
- Payments

Functionalities

- Generate patient bills
- Track payment status
- Record payment transactions

🔁 System Workflow

1. User registers and logs in
2. User books appointment with doctor
3. Doctor views assigned appointments
4. Doctor performs consultation

5. Medical record is created
6. Doctor generates prescription or requests lab tests
7. System generates bill
8. Patient makes payment
9. Patient views appointment history and treatment details

◈ Functional Requirements

- User authentication system
- Role-based login system
- Appointment booking system
- Appointment management
- Doctor management by admin
- Medical record management
- Prescription generation
- Lab test management
- Billing and payment system
- Secure data storage
- API-based communication
- Database connectivity

◈ Non-Functional Requirements

- User-friendly interface
- Secure authentication mechanism
- Reliable system performance
- Scalable database design
- Modular system architecture

- Maintainable code structure
- Data consistency and integrity
- Efficient system response time

rity